

# 目 录

|                |    |
|----------------|----|
| 动态规划 .....     | 3  |
| 背包问题 .....     | 3  |
| 01背包 .....     | 3  |
| 完全背包 .....     | 4  |
| 多重背包 .....     | 5  |
| 分组背包 .....     | 7  |
| 二维费用背包 .....   | 8  |
| 求具体方案 .....    | 9  |
| 求方案数 .....     | 11 |
| 有依赖的背包问题 ..... | 13 |
| 线性DP .....     | 14 |
| 数字三角形 .....    | 14 |
| 最长上升子序列 .....  | 16 |
| 最长公共子序列 .....  | 17 |
| 编辑距离 .....     | 19 |
| 区间DP .....     | 20 |
| 一维 .....       | 20 |
| 二维 .....       | 22 |
| 计数DP .....     | 23 |
| 数位DP .....     | 26 |
| 记忆化搜索 .....    | 27 |
| 试填法 .....      | 29 |
| 状压DP .....     | 30 |
| bitset优化 ..... | 33 |
| 重复覆盖问题 .....   | 34 |
| 树形DP .....     | 35 |
| 树上背包 .....     | 37 |
| 换根DP .....     | 38 |
| 概率DP .....     | 40 |
| 概率DP .....     | 41 |
| 期望DP .....     | 42 |
| 记忆化搜索 .....    | 43 |
| DP优化 .....     | 44 |

|              |    |
|--------------|----|
| 单调队列优化 ..... | 44 |
| 数据结构优化 ..... | 46 |
| 斜率优化 .....   | 49 |
| 滚动数组 .....   | 53 |
| 其他 .....     | 54 |
| 环形处理 .....   | 54 |
| 状态机DP .....  | 56 |

# 动态规划

[\[笔记\] DP从入门到入门 - Luckyblock - 博客园 \(cnblogs.com\)](#)

[【题单】动态规划（入门/背包/状态机/划分/区间/状压/数位/树形/数据结构优化） - 力扣 \(LeetCode\)](#)

1. 构造问题
2. 拆解子问题
3. 求解最简单子问题
4. 通过子问题推当前问题，构建状态转移方程
5. 判断复杂度

```
1  memset()初始化
2  dp[0][0][...] = 边界值
3  for(状态1 : 所有状态1的值){
4      for(状态2 : 所有状态2的值){
5          for(...){
6              //状态转移方程
7              dp[状态1][状态2][...] = 求max/min/sum...
8          }
9      }
10 }
```

## 背包问题

### 01背包

每个物品最多只能用一次

```
1  //一维优化
2  //dp[j] = max(dp[k], dp[k - v[i]] + w[i]);
3  //1. dp[i] 仅用到了dp[i-1]层，
4  //2. k与k-v[i] 均小于k
5  //3.若用到上一层的状态时,从大到小枚举，反之从小到大哦
6  #include <iostream>
7  #include <algorithm>
8  using namespace std;
9  const int N = 1003;
10 int n, m;
11 int v[N], w[N];
12 int dp[N]; //dp[i]表示背包容量不超过i下的最大价值
13
14 int main() {
```

```

15     cin >> n >> m;
16     for (int i = 0; i < n; i++) {
17         cin >> v[i] >> w[i];
18     }
19     for (int i = 0; i < n; i++) { //枚举所有物品
20         for (int k = m; k >= v[i]; k--) { //k从m~v[i]能枚举所有体积
21             dp[k] = max(dp[k], dp[k - v[i]] + w[i]);
22         }
23     }
24     cout << dp[m];
25     return 0;
26 }

```

```

1 //二维未优化版
2 int dp[N][N]; //dp[i][j]表示前i个物品,总体积不超过j的最大价值
3
4 for(int i = 1; i <= n; i++){
5     for(int j = 1; j <= m; j++){
6         if(j-v[i] >= 0) dp[i][j] = max(dp[i-1][j], dp[i-1][j-v[i]]+w[i]);
7         else dp[i][j] = dp[i-1][j];
8     }
9 }

```

## 完全背包

每个物品可以使用无数次

[完全背包问题 - AcWing题库](#)

```

1 //未优化版 时间复杂度较高 最大为O(nmk)
2 //f[i][j] = max(f[i][j], f[i-1][j-k*v[i]] + k*w[i]);

```

```

1 //空间二维优化 (i:1~n) (j:0~m)
2 if (j >= v[i]) f[i][j] = max(f[i-1][j], f[i][j-v[i]] + w[i]); //与01背包不同,这里从f[i]更新
3 else f[i][j] = f[i-1][j];

```

```

1 //空间一维优化 O(NM)
2 #include <iostream>
3 using namespace std;
4 const int N = 1003;
5 int n, m;
6 int w[N], v[N], dp[N];

```

```

7
8 int main(){
9     cin >> n >> m;
10    for(int i = 1; i <= n; i++){
11        cin >> v[i] >> w[i];
12    }
13    for(int i = 1; i <= n; i++){
14        for(int k = v[i]; k <= m; k++){//一维完全背包这里k从小到大枚举
15            dp[k] = max(dp[k], dp[k-v[i]]+w[i]);
16        }
17    }
18    cout << dp[m] << endl;
19    return 0;
20 }

```

## 多重背包

每个物品使用有限次

```

1 //https://www.acwing.com/problem/content/4/
2 //二维未优化 O(nms)
3 #include <iostream>
4 #include <algorithm>
5 using namespace std;
6 const int N = 103;
7 int n, m;
8 int v[N], w[N], s[N];
9 int f[N][N];
10
11 int main() {
12     cin >> n >> m;
13     for (int i = 1; i <= n; i++) {
14         cin >> v[i] >> w[i] >> s[i];
15     }
16     for (int i = 1; i <= n; i++) {
17         for (int j = 0; j <= m; j++) { //j: 0~m
18             for (int k = 0; k <= s[i] && k * v[i] <= j; k++) { //k*v[i] <= j
19                 f[i][j] = max(f[i][j], f[i-1][j-k*v[i]] + k*w[i]); //
20             }
21         }
22     }
23     cout << f[n][m];
24     return 0;
25 }

```

```

1 //https://www.acwing.com/problem/content/5/
2 //二进制优化 O(nmlog s) 再用01背包问题求解
3 //任意一个正整数s都可拆成1+2+4+8+16+... = s
4 #include <iostream>

```

```

5  using namespace std;
6  const int N = 200005;
7  int n,m;
8  int v[N],w[N],dp[N],idx;
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1;i <= n;i++){
13         int a,b,s;cin >> a >> b >> s;
14         int k = 1;
15         while(k <= s){
16             idx++;//第cnt个物品体积为a*s, 价值为b*s
17             v[idx] = a*k;
18             w[idx] = b*k;
19             s-=k;
20             k*=2;
21         }
22         if(s > 0){//补上剩下的s个物品
23             idx++;
24             v[idx] = a*s;
25             w[idx] = b*s;
26         }
27     }
28     for(int i = 1;i <= idx;i++){//再用01背包处理该idx个物品
29         for(int k = m;k >= v[i];k--){
30             dp[k] = max(dp[k],dp[k-v[i]]+w[i]);
31         }
32     }
33     cout << dp[m];
34 }
35
36 ////////////////////////////////////////////////////
37 //简写, 优化空间
38 #include <iostream>
39 using namespace std;
40 const int N = 2003;
41 int dp[N];
42
43 int main(){
44     int n,m;cin >> n >> m;
45     for(int i = 1;i <= n;i++){
46         int v,w,s; cin >> v >> w >> s;
47         for(int k = 1;k <= s;k <= 1){
48             s -= k;
49             for(int j = m;j >= k*v;j--){
50                 dp[j] = max(dp[j],dp[j-k*v]+k*w);
51             }
52         }
53         if(s){
54             for(int j = m;j >= s*v;j--){
55                 dp[j] = max(dp[j],dp[j-s*v]+s*w);
56             }

```

```

57     }
58 }
59 cout << dp[m];
60 }

```

```

1  //单调队列优化 O(nm)
2  #include <iostream>
3  #include <cstring>
4  using namespace std;
5  const int N = 1003,M = 20004;
6  int n,m;
7  int v[N],w[N],s[N];
8  int dp[M],g[M];//对于第i层只用到第i-1层,可用拷贝/滚动数组优化
9  int q[M],hh,tt = -1;//单调队列滑动窗口
10
11 int main(){
12     cin >> n >> m;
13     for(int i = 1;i <= n;i++){
14         cin >> v[i] >> w[i] >> s[i];
15     }
16     for(int i = 1;i <= n;i++){
17         memcpy(g,dp,sizeof g);//g[]作为dp[]的i-1层
18         int k = (s[i]+1)*v[i];//(s[i]+1)*v[i]为窗口最大长度, 诺超出则窗口右移
19         for(int r = 0;r < v[i];r++){//r为m%v[i]的偏移量
20             hh = 0,tt = -1;//L(hh),R(tt)
21             for(int j = r;j <= m;j += v[i]){
22                 if(hh <= tt && j-k >= q[hh]) hh++;
23                 while(hh <= tt && g[q[tt]]+(j-q[tt])/v[i]*w[i] <= g[j]) tt--;
24                 //把以队尾q[tt]为值时<=g[j]的数都弹出, 保持队列单调递减
25                 q[++tt] = j; //队首q[hh]为滑动窗口最大值
26                 dp[j] = g[q[hh]] + (j-q[hh])/v[i]*w[i];
27                 //dp[i][j] = dp[i-1][mx] + (j-mx)/v[i]*w[i];
28             }
29         }
30     }
31     cout << dp[m];
32 }

```

## 分组背包

每组有若干个物品，同一组内的物品最多只能选一个。

```

1  //https://www.acwing.com/problem/content/description/9/
2  #include <iostream>
3  using namespace std;

```

```

4  const int N = 105;
5  int n, m;
6  int dp[N]; //只从前i组物品中选，当前体积小于等于j的最大值
7  int v[N][N], w[N][N], s[N];
8  int main() {
9      cin >> n >> m;
10     for (int i = 1; i <= n; i++) {
11         cin >> s[i]; //每组s[i]个物品
12         for (int j = 1; j <= s[i]; j++) {
13             cin >> v[i][j] >> w[i][j];
14         }
15     }
16
17     for (int i = 1; i <= n; i++) { //枚举所有背包
18         for (int k = m; k >= 0; k--) { //从m~0枚举所有体积
19             for (int j = 0; j <= s[i]; j++) { //再枚举所有选择
20                 if (k >= v[i][j]) {
21                     dp[k] = max(dp[k] /*不选*/, dp[k - v[i][j]] + w[i][j] /*选*/);
22                 }
23             }
24         }
25     }
26     cout << dp[m];
27 }

```

## 二维费用背包

```

1  //二维费用01背包 https://www.acwing.com/problem/content/8/
2  //容量不超过V，重量不超过M
3  #include <iostream>
4  using namespace std;
5  const int MX = 1003;
6  int N, V, M;
7  int v[MX], m[MX], w[MX];
8  int dp[105][105];
9
10 int main(){
11     cin >> N >> V >> M;
12     for(int i = 1; i <= N; i++){
13         cin >> v[i] >> m[i] >> w[i];
14     }
15     for(int i = 1; i <= N; i++){
16         for(int j = V; j >= v[i]; j--){
17             for(int k = M; k >= m[i]; k--){
18                 dp[j][k] = max(dp[j][k], dp[j - v[i]][k - m[i]] + w[i]);
19             }
20         }
21     }
22     cout << dp[V][M];

```



```

1 //潜水员 http://ybt.ssoier.cn:8088/problem\_show.php?pid=1271
2 //01背包,求满足氧气>=A,氮气>=B的同时,最小重量和
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 int A,B,n;
7 int dp[25][80];
8
9 int main(){
10     memset(dp,0x3f,sizeof dp);
11     dp[0][0] = 0;
12     cin >> A >> B >> n;
13     for(int i = 1;i <= n;i++){
14         int a,b,w;cin >> a >> b >> w;
15         for(int j = A;j >= 0;j--){
16             for(int k = B;k >= 0;k--){
17                 dp[j][k] = min(dp[j][k],dp[max(j-a,0)][max(k-b,0)]+w);
18             }
19         }
20     }
21     cout << dp[A][B];
22 }
23 //空间未优化版 dp[i][j][k] = min(dp[i-1][j][k],dp[i-1][max(j-a,0)][max(k-b,0)]+w);

```

## 求具体方案

求具体方案，空间一般不能优化，方便正确从终点转移回溯

```

1 //01背包字典序最小的具体方案 https://www.acwing.com/problem/content/12/
2 //无法优化为一维
3 #include <iostream>
4 using namespace std;
5 const int N = 1003;
6 int n,m;
7 int v[N],w[N];
8 int dp[N][N];
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1;i <= n;i++){
13         cin >> v[i] >> w[i];
14     }
15
16     for(int i = n;i >= 1;i--){ //从后往前dp
17         for(int j = 0;j <= m;j++){

```

```

18         if(j >= v[i]) dp[i][j] = max(dp[i+1][j],dp[i+1][j-v[i]]+w[i]);
19         else dp[i][j] = dp[i + 1][j];
20     }
21 }
22
23 //dp[1][m]为终点状态
24 for(int i = 1,j = m;i <= n;i++){//再从前往后回溯,保证字典序最小
25     if(j >= v[i] && dp[i][j] == dp[i+1][j-v[i]]+w[i]){
26         cout << i << ' ';
27         j -= v[i];
28     }
29 }
30 }

```

```

1 //分组背包具体方案 机器分配:https://www.acwing.com/problem/content/1015/
2 #include <iostream>
3 using namespace std;
4 const int N = 20;
5 int n,m;
6 int w[N][N];
7 int dp[N][N];
8 int ans[N];
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1;i <= n;i++){
13         for(int j = 1;j <= m;j++){
14             cin >> w[i][j];
15         }
16     }
17
18     for(int i = 1;i <= n;i++){
19         for(int k = 0;k <= m;k++){
20             for(int j = 0;j <= m;j++){
21                 if(k >= j){
22                     dp[i][k] = max(dp[i][k],dp[i-1][k-j]+w[i][j]);
23                 }
24             }
25         }
26     }
27
28     cout << dp[n][m] << '\n';
29     for(int i = n,k = m;i >= 1;i--){
30         for(int j = 1;j <= m;j++){
31             if(k >= j && dp[i][k] == dp[i-1][k-j]+w[i][j]){
32                 k -= j;
33                 ans[i] = j;
34                 break;
35             }

```

```

36     }
37 }
38 for(int i = 1; i <= n; i++){//第i组里选第ans[i]个物品(可能为0)
39     cout << i << ' ' << ans[i] << '\n';
40 }
41 }

```

## 求方案数

```

1 //01背包求最优方案的方案数 https://www.acwing.com/problem/content/11/
2 //使总价值最大的最优方案数 O(nm)
3 #include <iostream>
4 using namespace std;
5 const int N = 1003, mod = 1e9+7;
6 int n, m;
7 int v[N], w[N];
8 int dp[N], f[N];
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1; i <= n; i++){
13         cin >> v[i] >> w[i];
14     }
15
16     for(int i = 0; i <= m; i++) { //什么都不选也是一种方案
17         f[i] = 1;
18     }
19
20     for(int i = 1; i <= n; i++){
21         for(int j = m; j >= v[i]; j--){
22             if(dp[j] < dp[j-v[i]]+w[i]){
23                 dp[j] = dp[j-v[i]]+w[i];
24                 f[j] = f[j-v[i]];
25             }
26             else if(dp[j] == dp[j-v[i]]+w[i]){
27                 f[j] = (f[j]+f[j-v[i]])%mod;
28             }
29         }
30     }
31     cout << f[m];
32 }

```

```

1 //01背包求价值恰好为m的方案数
  https://www.acwing.com/problem/content/description/280/
2 //O(nm)

```

```

3  #include <iostream>
4  using namespace std;
5  const int N = 105,M = 10004;
6  int n,m;
7  int a[N];
8  int dp[M];
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1;i <= n;i++){
13         cin >> a[i];
14     }
15
16     dp[0] = 1;
17     for(int i = 1;i <= n;i++){
18         for(int j = m;j >= a[i];j--){
19             dp[j] += dp[j-a[i]];
20         }
21     }
22     cout << dp[m];
23 }

```

```

1  //完全背包求最优的方案数(未验证是否正确)  O(nm)
2  #include <iostream>
3  #include <cstring>
4  using namespace std;
5  const int N = 1003,mod = 1e9+7;
6  int n,m;
7  int v[N],w[N];
8  int dp[N],f[N];
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1;i <= n;i++){
13         cin >> v[i] >> w[i];
14     }
15
16     for(int i = 0;i <= m;i++){
17         f[i] = 1;
18     }
19     for(int i = 1;i <= n;i++){
20         for(int j = v[i];j <= m;j++){
21             if(dp[j] < dp[j-v[i]]+w[i]){
22                 dp[j] = dp[j-v[i]]+w[i];
23                 f[j] = f[j-v[i]];
24             }
25             else if(dp[j] == dp[j-v[i]]+w[i]){
26                 f[j] = (f[j]+f[j-v[i]])%mod;

```

```

27     }
28     }
29 }
30 cout << f[m];
31 }

```

```

1 //完全背包求价值恰好为m的方案数 https://www.acwing.com/problem/content/1023/
2 // 给你一个n种面值的货币系统，求组成面值为m的货币有多少种方案。 O(nm)
3 #include <iostream>
4 using namespace std;
5 const int N = 20,M = 3003;
6 int n,m;
7 int a[N];
8 long long dp[M];
9
10 int main(){
11     cin >> n >> m;
12     for(int i = 1;i <= n;i++){
13         cin >> a[i];
14     }
15     dp[0] = 1;
16     for(int i = 1;i <= n;i++){
17         for(int j = a[i];j <= m;j++){
18             dp[j] += dp[j-a[i]];
19         }
20     }
21     cout << dp[m];
22 }

```

## 有依赖的背包问题

有  $N$  个物品和一个容量是  $V$  的背包。物品之间具有依赖关系，且依赖关系组成一棵树的形状。如果选择一个物品，则必须选择它的父节点。

```

1 //树上分组背包 dp与类似于洛谷P2014选课
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 105;
6 int n,m;
7 int v[N],w[N],p[N];
8 int h[N],e[N],ne[N],idx;
9 int dp[N][N];
10
11 void add(int a,int b){
12     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
13 }
14
15 void dfs(int u){

```

```

16     for(int i = h[u]; ~i; i = ne[i]){ //物品组
17         dfs(e[i]);
18         for(int j = m; j >= 0; j--){ //体积
19             for(int k = 0; k <= j - v[u]; k++){ //决策
20                 dp[u][j] = max(dp[u][j], dp[u][j - k] + dp[e[i]][k]);
21             }
22         }
23     }
24 }
25
26 int main(){
27     memset(h, -1, sizeof h);
28     cin >> n >> m;
29     int root;
30     for(int i = 1; i <= n; i++){
31         cin >> v[i] >> w[i] >> p[i];
32         for(int j = v[i]; j <= m; j++) dp[i][j] = w[i]; //初始状态
33         if(p[i] == -1) root = i;
34         else add(p[i], i);
35     }
36     dfs(root);
37     cout << dp[root][m];
38 }

```

## 线性DP

### 数字三角形

```

1 //https://www.acwing.com/problem/content/900/
2 #include <iostream>
3 using namespace std;
4 const int INF = 0x3f3f3f3f;
5 const int N = 505;
6 int arr[N][N], dp[N][N];
7 int n;
8
9 int main() {
10     cin >> n;
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= i; j++) {
13             cin >> arr[i][j];
14         }

```



```

28         x = max(x, dp[k-1][i1-1][i2] + t);
29         x = max(x, dp[k-1][i1][i2-1] + t);
30         x = max(x, dp[k-1][i1][i2] + t);
31     }
32 }
33 }
34 }
35 cout << dp[n+n][n][n];
36 }

```

## 最长上升子序列

**LIS** Longest Increasing Subsequence, 最长递增子序列

```

1 //https://www.luogu.com.cn/problem/B3637
2 //O(n^2) -- DP
3 #include <iostream>
4 using namespace std;
5 const int N = 5005;
6 int n, ans;
7 int arr[N], dp[N]; //dp[i]为以第i个点的a[i]结尾的最大的上升序列
8
9 int main() {
10     cin >> n;
11     for (int i = 1; i <= n; i++) {
12         cin >> arr[i];
13     }
14
15     for (int i = 1; i <= n; i++) {
16         dp[i] = 1; //初始化dp[i] = 1;
17         for (int j = 1; j < i; j++) {
18             if (arr[i] > arr[j]) {
19                 dp[i] = max(dp[i], dp[j] + 1);
20                 //前一个小于自己的数结尾的最大上升子序列加上自己, 即+1
21             }
22         }
23         ans = max(ans, dp[i]);
24     }
25     cout << ans;
26 }

```

```

1 //https://www.acwing.com/problem/content/898/
2 //O(nlogn) -- 模拟栈 + 二分
3 //诺求最长单调非递减子序列 1b改为ub, >改>=即可:
4 //https://ac.nowcoder.com/acm/contest/83252/D
5 #include <iostream>
6 #include <vector>

```



```

6   using namespace std;
7
8   const int N = 100005;
9   int n, arr[N];
10  vector<int> v; //模拟堆栈
11
12  int main() {
13      cin >> n;
14      for (int i = 1; i <= n; i++) {
15          cin >> arr[i];
16      }
17
18      for (int i = 1; i <= n; i++) {
19          if (v.empty() || arr[i] > v.back()) { //如果arr[i]大于栈顶元素,将该元素入栈
20              v.push_back(arr[i]);
21          }
22          else { //否则替换掉第栈中一个大于或者等于arr[i]的那个数
23              *lower_bound(begin(v), end(v), arr[i]) = arr[i];
24          }
25      }
26      cout << v.size();
27  }

```

## 最长公共子序列

**LCS** Longest Common Subsequence, 最长公共子序列

```

1   //https://www.acwing.com/problem/content/899/
2   //O(nm)
3   #include <iostream>
4   using namespace std;
5   const int N = 1005;
6   int n, m, dp[N][N]; //dp[i][j]表示a的前i个字母和b的前j个字母最长公共子序列长度
7   string a, b;
8   int main() {
9       cin >> n >> m >> a >> b;
10      a = ' ' + a, b = ' ' + b;
11      for (int i = 1; i <= n; i++) {
12          for (int j = 1; j <= m; j++) {
13              if (a[i] == b[j]) { //情况一: a[i]在b[j]在(dp[i][j])
14                  dp[i][j] = max(dp[i][j], dp[i-1][j-1]+1);
15              }
16              else { //情况二: a[i]在b[j]不在(dp[i][j-1])、情况三: a[i]不在b[j]在(dp[i-1][j])
17                  //情况四: a[i]不在b[j]不在(dp[i-1][j-1]),已经包含在情况二、三中
18                  dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
19              }
20          }
21      }
22  }

```

```

21     }
22     cout << dp[n][m];
23     /* 具体序列方案
24     string s;
25     for(int i = n,j = m;i >= 1 && j >= 1;){
26         if(dp[i][j] == dp[i-1][j-1]+1){
27             s += s1[i];
28             i--;j--;
29         }
30         else if(dp[i][j] == dp[i-1][j]) i--;
31         else if(dp[i][j] == dp[i][j-1]) j--;
32     }
33     for(int i = s.size()-1;i >= 0;i--){
34         cout << s[i];
35     }*/
36 }

```

## 最长公共上升子序列LCIS

```

1 //https://www.acwing.com/problem/content/274/
2 #include <iostream>
3 using namespace std;
4 const int N = 3003;
5 int n;
6 int a[N],b[N];
7 int dp[N][N];/*a[]中前i个数字,b[]中前j个数字,...且当前以b[j]结尾的子序列的最长方案
8
9 int main(){
10     cin >> n;
11     for(int i = 1;i <= n;i++) cin >> a[i];
12     for(int i = 1;i <= n;i++) cin >> b[i];
13
14     for(int i = 1;i <= n;i++){
15         int nmax = 1;
16         for(int j = 1;j <= n;j++){
17             dp[i][j] = dp[i-1][j];
18             if(b[j] == a[i]) dp[i][j] = max(dp[i][j],nmax);
19             if(b[j] < a[i]) nmax = max(nmax,dp[i-1][j] + 1);
20         }
21     }
22
23     int ans = 0;
24     for(int i = 0;i <= n;i++){ans = max(ans,dp[n][i]);}
25     cout << ans;
26     return 0;
27 }

```

## 编辑距离

将字符串a通过以下操作变为字符串b的最小操作次数

1. 删除一个字符
2. 插入一个字符
3. 修改一个字符

```
1  if (a[i] == b[j]) dp[i][j] = dp[i-1][j-1];    //无需修改
2  else{
3      dp[i][j] = min({dp[i-1][j-1]+1,dp[i-1][j]+1,dp[i][j-1]+1});
4      /*min{
5          a[1~i-1] = b[1~j-1],修改a[i]为b[j]
6          a[1~i-1] = b[1~j],删除a[i]
7          a[1~i] = b[1~j-1],增加b[j]
8      }*/
9  }
```

```
1  //https://www.luogu.com.cn/problem/P2758
2  #include <iostream>
3  #include <algorithm>
4  using namespace std;
5  const int N = 2003;
6  int n,m;
7  string a,b;
8  int dp[N][N]; //dp[i][j]为将a[1~i]变为b[1~j]的最小操作次数
9
10 int main(){
11     cin >> a >> b; n = a.size(); m = b.size();
12     a = ' ' + a, b = ' ' + b;
13
14     for(int i = 1; i <= n; i++) dp[i][0] = i; //初始化
15     for(int j = 1; j <= m; j++) dp[0][j] = j;
16
17     for(int i = 1; i <= n; i++){
18         for(int j = 1; j <= m; j++){
19             if(a[i] == b[j]){
20                 dp[i][j] = dp[i-1][j-1];
21             }
22             else{
23                 dp[i][j] = min({dp[i-1][j-1]+1,dp[i-1][j]+1,dp[i][j-1]+1});
24             }
25         }
26     }
27     cout << dp[n][m];
28 }
```

# 区间DP

## 定义

区间类动态规划是线性动态规划的扩展，它在分阶段地划分问题时，与阶段中元素出现的顺序和由前一阶段的哪些元素合并而来有很大的关系。

令状态  $f(i, j)$  表示将下标位置  $i$  到  $j$  的所有元素合并能获得的价值最大值，那么  $f(i, j) = \max\{f(i, k) + f(k + 1, j) + cost\}$ ,  $cost$  为将这两组元素合并起来的价值。

## 性质

区间 DP 有以下特点：

合并：即将两个或多个部分进行整合，当然也可以反过来；

特征：能将问题分解为能两两合并的形式；

求解：对整个问题设最优值，枚举合并点，将问题分解为左右两个部分，最后合并两个部分的最优值得到原问题的最优值。

## 一维

### 石子合并

$n$  个数  $a[1] \sim a[n]$  排成一排，进行  $n-1$  次合并操作，每次操作将相邻的两堆合并成一堆，并获得新的一堆中的石子数量的和的得分。你需要最小化你的得分。

诺要求环形的，只需将两个  $a[]$  接在一起。枚举所有长度为  $n$  的区间取最值

```
1 //https://www.luogu.com.cn/problem/P1775
2 //O(n^3)
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 305;
7 int n;
8 int s[N]; //a[] 的前缀和
9 int dp[N][N]; //dp[i][j] 表示将i到j这一段石子合并成一堆的方案集合，属性Min
10
11 int main() {
12     cin >> n;
13     for (int i = 1; i <= n; i++) {
14         cin >> s[i];
15         s[i] += s[i-1];
16     }
17
18     memset(dp, 0x3f, sizeof dp);
19     for (int i = 1; i <= n; i++) dp[i][i] = 0;
20
21     for (int len = 2; len <= n; len++) { //枚举区间长度，可以跳过1，从2开始
22         for (int l = 1; l + len - 1 <= n; l++) { //枚举区间左右端点
```

```

23         int r = l + len - 1;
24         for (int k = l; k < r; k++) { // 枚举分割点, 构造状态转移方程
25             dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r] + s[r] - s[l-1]);
26         }
27     }
28 }
29 cout << dp[1][n];
30 }
31 ///////////////////////////////////////////////////
32 //记忆化搜索写法
33 int sol(int l, int r){
34     if(l >= r) return 0;
35     if(dp[l][r] != -1) return dp[l][r];
36     dp[l][r] = 0x3f3f3f3f;
37     for(int k = l; k <= r-1; k++){
38         dp[l][r] = min(dp[l][r], sol(l, k) + sol(k+1, r) + s[r] - s[l-1]);
39     }
40     return dp[l][r];
41 }
42
43 memset(dp, -1, sizeof dp);
44 cout << sol(1, n);

```

当n很大时, 用Garsia-Wachs 算法求解

每次找到第一个 $a[i-1] < a[i+1]$ 的位置, 把 $a[i-1]$ 和 $a[i]$ 合并为now, 再将now插入到从i开始左边第一个 $a[k] > now$ 的 $a[k]$ 后面

```

1 //https://www.luogu.com.cn/problem/P5569
2 //需要开启O2优化, 或使用平衡树等数据结构
3 #include<bits/stdc++.h>
4 using namespace std;
5 const int N = 40004;
6 const long long INF = 1e18;
7 long long ans;
8
9 int main(){
10     int n; cin >> n;
11     vector<long long> v(n+2);
12     for(int i = 1; i <= n; i++){ cin >> v[i]; }
13     v[0] = v[n+1] = INF; //在两端设置哨兵
14     n++;
15
16     while(n-- && n >= 2){
17         int i, k;
18         for(i = 1; i <= n; i++){
19             if(v[i-1] < v[i+1]) break;
20         }
21         int now = v[i-1] + v[i];
22         ans += now;

```

```

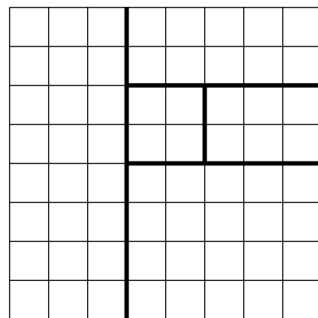
23     for(k = i-1;k >= 0;k--){
24         if(v[k] > now) break;
25     }
26
27     v.erase(v.begin()+i-1);
28     v.erase(v.begin()+i-1);
29     v.insert(v.begin()+k+1,now);
30 }
31 cout << ans;
32 }

```

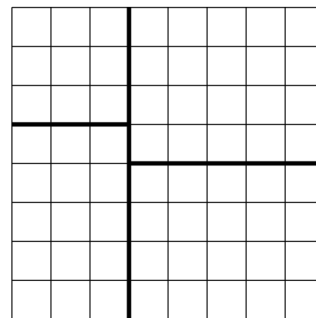
## 二维

### 棋盘分割

将一个  $8 \times 8$  的棋盘进行如下分割：将原棋盘割下一块矩形棋盘并使剩下部分也是矩形，再将剩下的两部分中的任意一块继续如此分割，这样割了  $(n - 1)$  次后，连同最后剩下的矩形棋盘共有  $n$  块矩形棋盘。（每次切割都只能沿着棋盘格子的边进行）。原棋盘上每一格有一个分值，一块矩形棋盘的总分为其所含各格分值之和。现在需要把棋盘按上述规则分割成  $n$  块矩形棋盘，并使各矩形棋盘总分的平方和最小，给定  $n$  求最小值时多少？



允许的分割方案



不允许的分割方案

```

1  #include <iostream>
2  #include <cmath>
3  #include <cstring>
4  using namespace std;
5  const long long INF = 1e18;
6  const int N = 20,M = 10;
7  int n,m = 8;
8  long long a[M][M],s[M][M];
9  long long dp[M][M][M][M][N];
10 //dp[x1][y1][x2][y2][k]表示该矩阵且切割了k次的最小平方和
11
12 long long get(int x1,int y1,int x2,int y2){
13     long long sum = s[x2][y2] - s[x2][y1-1] - s[x1-1][y2] + s[x1-1][y1-1];
14     return sum*sum;

```

```

15 }
16
17 long long dfs(int x1,int y1,int x2,int y2,int k){//记忆化搜索dp
18     long long &now = dp[x1][y1][x2][y2][k];
19     if(now >= 0) return now;//如果该区间已经求过，直接返回
20     if(k == 1) return now = get(x1,y1,x2,y2);//k=1不分割，直接返回该区间和
21     now = INF;
22     for(int i = x1;i < x2;i++){//枚举横着分割
23         now = min(now,dfs(x1,y1,i,y2,k-1)+get(i+1,y1,x2,y2));//选上半段
24         now = min(now,dfs(i+1,y1,x2,y2,k-1)+get(x1,y1,i,y2));//选下半段
25     }
26     for(int i = y1;i < y2;i++){//枚举竖着分割
27         now = min(now,dfs(x1,y1,x2,i,k-1)+get(x1,i+1,x2,y2));//选左半段
28         now = min(now,dfs(x1,i+1,x2,y2,k-1)+get(x1,y1,x2,i));//选右半段
29     }
30     return now;
31 }
32
33 int main(){
34     cin >> n;
35     for(int i = 1;i <= m;i++){
36         for(int j = 1;j <= m;j++){
37             cin >> a[i][j];
38             s[i][j] += s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j];
39         }
40     }
41     memset(dp,-0x3f,sizeof dp);
42     cout << dfs(1,1,8,8,n);
43 }

```

## 计数DP

### 整数划分

一个正整数  $n$  可以表示成若干个正整数之和，形如： $n=n_1+n_2+\dots+n_k$ ，其中  $n_1\geq n_2\geq\dots\geq n_k, k\geq 1$ 。现在给定一个正整数  $n$  ( $1\leq n\leq 1000$ )，请你求出  $n$  共有多少种不同的划分方法。

完全背包解法：把1,2,3, ...  $n$ 分别看做 $n$ 个物体的体积，这 $n$ 个物体均无使用次数限制，问恰好能装满总体积为 $n$ 的背包的总方案数

```

1 //https://www.acwing.com/problem/content/902/
2 //O(N^2)
3 #include <iostream>
4 using namespace std;
5 const int N = 1005, mod = 1e9 + 7;

```

```

6  int n, dp[N];
7
8  int main() {
9      cin >> n;
10
11     dp[0] = 1; //容量为0时，前i个物品全不选也是一种方案
12
13     for (int i = 1; i <= n; i++) {
14         for (int j = i; j <= n; j++) {
15             dp[j] = (dp[j] + dp[j - i]) % mod;
16         }
17     }
18     cout << dp[n];
19 }

```

另一种解法：dp[i][j]表示为总和为i，总个数为j的方案数

```

1  #include <iostream>
2  using namespace std;
3  const int N = 1005, mod = 1e9 + 7;
4  int n, ans, dp[N][N];
5
6  int main() {
7      cin >> n;
8
9      dp[0][0] = 1;
10
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= i; j++) {
13             dp[i][j] = (dp[i - 1][j - 1] + dp[i - j][j]) % mod;
14         }
15     }
16
17     for (int i = 1; i <= n; i++) {
18         ans = (ans + dp[n][i]) % mod;
19     }
20
21     cout << ans;
22 }

```

<https://codeforces.com/contest/560/problem/E>

给定一个H\*W的棋盘，其中有n个黑格子不能走，每次只能向下或右走，问从左上走到右下角共有多少种走法？

排序后，令dp[i]表示从左上走到排序后的第i个格子且不经过其他黑格子的走法，则有：

$$dp[i] = C_{x_i+y_i-2}^{x_i-1} - \sum_{j=0}^{i-1} dp[j] * C_{x_i+y_i-x_j-y_j}^{x_i-x_j}, \text{ 其中 } x_i \geq x_j \text{ 且 } y_i \geq y_j$$



```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int N = 200005;
5  int h,w,n;
6  const int mod = 1e9+7;
7  long long dp[N];
8
9  struct node{
10     int x,y;
11     bool operator < (const auto&e2){
12         if(x != e2.x) return x < e2.x;
13         return y < e2.y;
14     }
15 }a[N];
16
17 long long qmi(long long a,long long b,long long p){
18     long long ans = 1;
19     while(b){
20         if(b&1) ans = ans*a%mod;
21         b >>= 1;
22         a = a*a%mod;
23     }
24     return ans%mod;
25 }
26
27 long long fact[N],infact[N];
28 void init(){
29     fact[0] = infact[0] = 1;
30     for(int i = 1;i < N;i++){
31         fact[i] = i*fact[i-1]%mod;
32     }
33     infact[N-1] = qmi(fact[N-1],mod-2,mod);
34     for(int i = N-2;i >= 1;i--){
35         infact[i] = infact[i+1]*(i+1)%mod;
36     }
37 }
38
39 long long C(int a,int b){
40     return fact[a]*infact[b]%mod*infact[a-b]%mod;
41 }
42
43 int main(){
44     init();
45     cin >> h >> w >> n;
46     for(int i = 1;i <= n;i++){ cin >> a[i].x >> a[i].y; }
47     sort(a+1,a+n+1);
48     a[n+1] = {h,w};
49     for(int i = 1;i <= n+1;i++){
50         auto [x,y] = a[i];
51         dp[i] = C(x+y-2,x-1);
52         for(int j = 1;j < i;j++){

```

```

53         if(x < a[j].x || y < a[j].y) continue;
54         dp[i] -= dp[j]*C(x+y - a[j].x-a[j].y,x-a[j].x)%mod;
55     }
56     dp[i] = (dp[i]%mod+mod)%mod;
57 }
58 cout << dp[n+1];
59 }

```

## 数位DP

[数位 DP - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/dp/digit/)

求区间 $[l,r]$ 中答案的个数，可以转化为求  $F(r) - F(l-1)$ ，其中 $F(x)$ 为区间 $[1,x]$ 中答案的个数

[P2602 [ZJOI2010 数字计数 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P2602)]

题目大意：给定两个正整数 $a,b$ ,求在 $[a,b]$ 中的所有整数中，每个数码(digit)各出现了多少次。

```

1  #include <iostream>
2  #include <cstring>
3
4  const int N = 13;
5  int nums[N],len;
6  long long dp[N][N][10]; //dp[pos][sum]表示pos位置,当前已经统计了sum个目标数字的方案数
7
8  long long dfs(int pos,bool lim,bool zero,int dig,int sum){
9      if(!pos) return sum;
10     if(!lim && !zero && ~dp[pos][sum][dig]) return dp[pos][sum][dig];
11     int up = lim ? nums[pos] : 9;
12     long long ans = 0;
13     for(int i = 0; i <= up; i++){
14         bool count = (!(zero & !i)) && (i == dig); //当前i==dig且不存在前导0
15         ans += dfs(pos-1,lim & (i==up),zero & !i,dig,sum+count);
16     }
17     return (lim || zero) ? ans : dp[pos][sum][dig] = ans;
18 }
19
20 long long f(long long x,int dig){
21     len = 0;
22     while(x) nums[++len] = x % 10, x /= 10;
23     return dfs(len,1,1,dig,0);
24 }
25
26 int main(){
27     long long l,r; std::cin >> l >> r;

```

```

28     std::memset(dp,-1,sizeof dp);
29     for(int i = 0;i <= 9;i++){
30         std::cout << f(r,i) - f(l-1,i) << ' ';
31     }
32 }

```

## 记忆化搜索

数位DP建议使用记忆化搜索做，大部分情况下比递推简单一点。

update(state): 123\_\_->1234\_

```

1  //模版
2  //len从高为到低位开始处理
3  //lim表示当前数位是否有限制,初始值为1
4  //zero表示是否有前导零,诺前导零不影响答案则可以删除,初始值为1
5  //state可能需要设计多个
6  ll dp[N][state];
7  int nums[N],len;
8
9  ll dfs(int pos,bool lim,bool zero,int state){
10     if(!pos) return check(state); //已经处理到最后一位,返回最终状态是否满足条件
11     if(!lim && !zero && ~dp[pos][state]) return dp[pos][state]; //当前既没有限制,也没有前导零,且答案之前处理过则直接返回结果
12     int up = lim ? nums[pos] : 9; //up表示当前位上限,有限制则只能处理0~nums[pos],否则能处理0~9
13     ll ans = 0;
14     for(int i = 0;i <= up;i++){
15         ans += dfs(pos-1,lim&(i==up),zero&(!i),update(state, i));
16         //pos-1:处理下一位
17         //lim & (i == up)更新限制状态
18         //zero & (!i) 更新前导零状态
19         //update(state, i) 更新状态
20     }
21     return (lim || zero) ? ans : dp[pos][state] = ans; //如果当前有限制或有前导零则返回答案,否则将答案记忆化再返回
22 }
23
24 ll f(ll x){
25     len = 0;
26     while(x) nums[++len] = x % 10, x /= 10;
27     return dfs(len,1,1,0);
28 }
29
30 memset(dp, -1, sizeof dp);

```

```
31 cout << f(r) - f(l - 1);
```

### [XHXJ's LIS - HDU 4352 - Virtual Judge \(vjudge.net\)](#)

求区间[L, R]内有多少整数的最长上升子序列长度为K?

考虑到最长上升子序列值域为不会超过9, 可以用一个二进制状态压缩表示当前LIS的状态。前导0不能算入LIS, 所以参数要传入前导0状态。多开一维K可以节省初始化DP数组的时间

```
1  #include <iostream>
2  #include <cstring>
3
4  const int N = 22;
5  int k;
6  long long dp[N][1 << 10][10]; //dp[pos][state][k]表示第pos位,LIS状态压缩为state,LIS
   为k的方案数
7  int nums[N], len;
8
9  int update(int state, int x, int zero){
10     if(zero && (!x)) return state; //前导零不计入序列
11     for(int i = x; i <= 9; i++){ //用x替换state中第一个>=x的数
12         if(state >> i & 1) {
13             return state & (~ (1 << i)) | (1 << x);
14         }
15     }
16     return state | 1 << x;
17 }
18
19 long long dfs(int pos, int state, int lim, int zero){ //lim判断当前位是否有限制, zero判断
   是否有前导0
20     if(!pos) return __builtin_popcount(state) == k; //递归终点, 判断LIS长度是否为k
21     if(~dp[pos][state][k] && !lim) return dp[pos][state][k]; //记忆化结果(无限制时复
   用)
22     int up = lim ? nums[pos] : 9; //当前位上限, 无限制则可任意填
23     long long ans = 0;
24     for(int i = 0; i <= up; i++){
25         ans += dfs(pos-1, update(state, i, zero), lim && (i == up), zero && (!i));
26     }
27     return lim ? ans : dp[pos][state][k] = ans; //无限制时结果记忆化
28 }
29
30 long long f(long long x){
31     len = 0;
32     while(x) nums[++len] = x % 10, x /= 10;
33     return dfs(len, 0, 1, 1);
34 }
35
36 void sol(){
37     long long l, r; std::cin >> l >> r >> k;
38     std::cout << f(r) - f(l-1) << '\n';
```

```

39 }
40
41 int main(){
42     std::memset(dp,-1,sizeof dp);
43     int t; std::cin >> t;
44     for(int i = 1;i <= t;i++){
45         printf("Case #%d: ",i);
46         sol();
47     }
48 }

```

## 试填法

[P10958 启示录 - 洛谷 \(luogu.com.cn\)](#)

只要某数字的十进制表示中有三个连续的 6，即认为这是个魔鬼的数，比如 666,1666,6663,16666,6660666 等等。

给定n，求第n个魔鬼数是多少。

也可以使用二分+记忆化搜索：[记录详情 \(luogu.com.cn\)](#)

```

1  #include <iostream>
2  using namespace std;
3  const int N = 12;
4  int n;
5  int dp[N][3],g[N];//dp[i][j]表示共有i位,且有连续j(0~2)个6, g[i]表示i位共有多少个魔鬼数
6
7  void init(){
8      dp[0][0] = 1;
9      dp[0][1] = dp[0][2] = 0;
10     for(int i = 1; i < N;i++){
11         dp[i][0] = 9*(dp[i-1][0]+dp[i-1][1]+dp[i-1][2]);
12         dp[i][1] = dp[i-1][0];
13         dp[i][2] = dp[i-1][1];
14         g[i] = 10*g[i-1] + dp[i-1][2];
15     }
16 }
17
18 void sol(){
19     cin >> n;
20     int m = 3;
21     while(g[m] < n) m++;//确定位数,第n个魔鬼数有m位
22     for(int i = m,k = 0;i >= 1;i--){
23         for(int j = 0;j <= 9;j++){//当前第i位诺填j
24             long long cnt = g[i-1];//后面i-1位还有cnt种填法能让整个数是魔鬼数
25             if(j == 6 || k == 3){
26                 //当前位i开始,左边已经有了连续k+(j==6)个6, 还差3-(k+(j==6))个6
27                 for(int l = max(3-k-(j==6),0);l <= 2;l++){
28                     cnt += dp[i-1][l];

```

```

29         }
30     }
31     if(cnt >= n){
32         cout << j;
33         if(k < 3){
34             if(j == 6) k++;
35             else k = 0;
36         }
37         break;
38     }
39     else n -= cnt;
40 }
41 }
42 cout << '\n';
43 }
44
45 int main(){
46     init();
47     int t; cin >> t;
48     while(t--) sol();
49 }

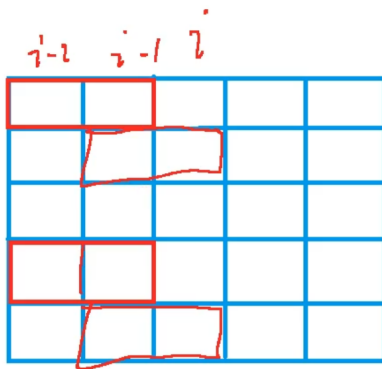
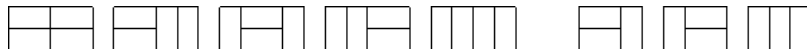
```

## 状压DP

[状压 DP - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/dp/state/)

### 长方形摆放

求把  $N \times M$  的棋盘分割成若干个  $1 \times 2$  的长方形，有多少种方案。



用二进制表示第  $i$  列状态，如第 1 列状态为 10010，第 2 列状态为 01001， $st[1][2] = 11011$

```

1 //https://www.acwing.com/problem/content/description/293/
2 #include <iostream>
3 #include <cstring>
4 #include <vector>
5 using namespace std;
6 const int N = 12, M = 1 << N; //M = 2^N

```

```

7  int n, m;
8  bool st[M];
9  long long dp[N][M]; //第一维表示列， 第二维表示所有可能的状态(1表示当前位置放了横着的长方形的左半边)
10 vector<int>v[M]; //二维数组记录合法的状态
11
12 int main() {
13     while (cin >> n >> m, n && m) { //n行, m列
14         //预处理1: 筛掉连续奇数个0的状态(不能竖着放满长方形)
15         for (int j = 0; j < (1 << n); j++) { //j < 2^n
16             st[j] = 1;
17             int cnt = 0; //cnt记录连续0的个数
18             for (int k = 0; k < n; k++) {
19                 if ((j >> k) & 1) { //如果当前位为1, 且前有连续奇数个0则状态不合法
20                     if (cnt & 1) st[j] = 0;
21                     cnt = 0;
22                 }
23                 else cnt++;
24             }
25             if (cnt & 1) st[j] = 0; //判断最后一节连续0个数
26         }
27
28         //预处理2: 看第i列能使用那些状态而不会和第i-1列冲突
29         for (int j = 0; j < (1 << n); j++) { //枚举第i列的所有状态j
30             v[j].clear(); //初始化清空上次操作遗留的状态, 防止影响本次状态。
31             for (int k = 0; k < (1 << n); k++) { //枚举第i-1列的所有状态k
32                 if ((j & k) == 0 && st[j | k]) {
33                     //j&k按位与判断i-1列伸到i列和第i列状态是否重合冲突
34                     //j|k按位或判断i-1列伸出去的方块是否会导致i列有连续奇数个0
35                     v[j].emplace_back(k);
36                     //j表示第i列的可行状态, k表示第i列状态为j的情况下第i-1列所有可行状态
37                 }
38             }
39         }
40
41         memset(dp, 0, sizeof dp);
42         dp[0][0] = 1;
43         //第1列由虚构的第0列推导, 第0列不可能放横条, 只能全摆薯条, 故摆法只有一种, 无需考虑奇偶性
44         for (int i = 1; i <= m; i++) {
45             for (int j = 0; j < (1 << n); j++) {
46                 for (auto& k : v[j]) {
47                     dp[i][j] += dp[i - 1][k];
48                 }
49             }
50         }
51         cout << dp[m][0] << endl;
52         //dp[m][0]表示 前m-1列都处理完, 并且第m-1列没有伸出来的所有方案数
53         //既整个棋盘处理完的方案数
54     }
55 }

```

## 最短Hamilton路径

给定一张  $n(n \leq 20)$  个点的带权无向图，点从  $0 \sim n-1$  标号，求 起点为  $0$  且 终点为  $n-1$  的最短Hamilton路径。Hamilton路径的定义是从  $0$  到  $n-1$  不重不漏地经过每个点恰好一次。

时间复杂度  $O(n^2 2^n)$

```
1 //https://ac.nowcoder.com/acm/problem/50909
2 #include <iostream>
3 #include <algorithm>
4 #include <cstring>
5 using namespace std;
6 const int N = 21, M = 1 << N;
7 int n;
8 int a[N][N]; //带权无向图
9 int dp[M][N]; //dp[i][j], i表示当前走过的点的集合, j表示当前停在了哪个点
10 //i为二进制表示路径, 如走0,1,2,4点则i为(10111)=23
11
12 int main() {
13     cin >> n;
14     for (int i = 0; i < n; i++) {
15         for (int j = 0; j < n; j++) {
16             cin >> a[i][j];
17         }
18     }
19     memset(dp, 0x3f, sizeof dp); //因为要求最小值, 所以初始化为无穷大
20     dp[1][0] = 0; //0为起点, 所以dp[1][0] = 0;
21
22     for (int i = 1; i < 1 << n; i++) { //枚举所有状态(已经经过的点)
23         for (int j = 0; j < n; j++) {
24             if (i >> j & 1) { //枚举当前停留的点j
25                 for (int k = 0; k < n; k++) {
26                     if (i >> k & 1) { //枚举上一个停留的点k
27                         dp[i][j] = min(dp[i][j], dp[i - (1 << j)][k] + a[k][j]);
28                         //从点k转移到点j
29                     }
30                 }
31             }
32         }
33     }
34     cout << dp[(1 << n) - 1][n - 1]; // [1111111...][n-1]表示所有点都走过, 最后停留的点
    为n-1
35 }
```



## bitset优化

[Many Graph Queries \(★7\) - AtCoder typical90 bg - Virtual Judge \(vjudge.net\)](#)

给定 $N$ 个点,  $M$ 条边的有向图。回答 $Q$ 个询问: 从点 $a$ 能否到达点 $b$ ?  $N, M, Q \leq 10^5$

将询问分成 $\sqrt{Q}$ 组, 依次处理每组询问。

```
1  #include <iostream>
2  #include <bitset>
3  #include <vector>
4  using namespace std;
5
6  int main(){
7      int n,m,q;cin >> n >> m >> q;
8      vector<vector<int>>>e(n);
9      for(int i = 0;i < m;i++){
10         int a,b;cin >> a >> b;
11         a--;b--;
12         e[a].emplace_back(b);
13     }
14
15     vector<int>a(q),b(q);
16     for(int i = 0;i < q;i++){
17         cin >> a[i] >> b[i];
18         a[i]--;b[i]--;
19     }
20
21     for(int i = 0;i < q;i += 316){
22         vector<bitset<316>>dp(n+1);
23         for(int j = 0;j < 316 && i+j < q;j++){
24             dp[a[i+j]][j] = 1;
25         }
26         for(int x = 0;x < n;x++){
27             for(auto& y:e[x]){
28                 dp[y] |= dp[x];
29             }
30         }
31         for(int j = 0;j < 316 && i+j < q;j++){
32             if(dp[b[i+j]][j]) cout << "Yes\n";
33             else cout << "No\n";
34         }
35     }
36 }
```

## 重复覆盖问题

给定 $n$ 个集合，和一个目标集合，选择最少的集合，使得所选集合的并集=目标集合

时间复杂度 $O(N * 2^M)$ ，若复杂度过高，可以考虑Dancing Links解决

```
1 //糖果 https://www.luogu.com.cn/problem/P8687
2 //dp[i]表示从状态0000走到状态i的最少步数
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 105,M = 1 << 21;
7 int a[N];
8 int dp[M];
9
10 int main(){
11     int n,m,k;cin >> n >> m >> k;
12     for(int i = 1;i <= n;i++){
13         for(int j = 1;j <= k;j++){
14             int x;cin >> x;
15             a[i] |= 1 << (x-1);
16         }
17     }
18     memset(dp,0x3f,sizeof dp);
19     dp[0] = 0;
20     for(int i = 1;i <= n;i++){
21         for(int j = 0;j < 1 << m;j++){
22             dp[a[i]|j] = min(dp[a[i]|j],dp[j]+1);//核心代码
23         }
24     }
25     int ans = dp[(1<<m)-1];
26     if(ans == 0x3f3f3f3f) cout << -1;
27     else cout << ans;
28 }
```

```
1 //dfs+记忆化写法 dp[i]表示从状态i走到目标状态1111的最少步数,如果dp[i]=0(且i!=n)则没有访问
  过该状态
2 //memset(dp,0,sizeof dp);
3 //int ans = dfs(0);
4 int dfs(int u){
5     if(u == (1 << m)-1) return 0;//目标状态
6     if(dp[u]) return dp[u];//若状态u到之前达过，则直接返回
7     int ans = 0x3f3f3f3f;//可以用dp[u] = 0x3f3f3f3f替换;
8     for(int i = 1;i <= n;i++){
9         if((u|a[i]) == u) continue;//只有状态发生改变才转移，防止死循环
10        ans = min(ans,dfs(u|a[i])+1);
11    }
12    return dp[u] = ans;
13 }
```

# 树形DP

## 洛谷 P1352 没有上司的舞会

求解一颗有向树的最大权独立集，每条边最多选一个点

我们设  $f(i, 0/1)$  代表以  $i$  为根的子树的最优解（第二维的值为 0 代表  $i$  不参加舞会的情况，1 代表  $i$  参加舞会的情况）。

对于每个状态，都存在两种决策（其中下面的  $x$  都是  $i$  的儿子）：

- 上司不参加舞会时，下属可以参加，也可以不参加，此时有
$$f(i, 0) = \sum \max\{f(x, 1), f(x, 0)\};$$
- 上司参加舞会时，下属都不会参加，此时有  $f(i, 1) = \sum f(x, 0) + a_i$ 。

我们可以通过 DFS，在返回上一层时更新当前结点的最优解。

```
1 //https://www.luogu.com.cn/problem/P1352
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 6006;
6 int n, w[N];
7 int h[N], e[N], ne[N], idx;
8 int dp[N][2]; // dp[i][0] 表示 i 节点不参加, dp[i][1] 表示 i 节点参加
9 bool st[N]; // 标记节点是否有父节点
10
11 void add(int a, int b) {
12     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
13 }
14
15 void dfs(int u) {
16     dp[u][1] = w[u];
17     for (int i = h[u]; i != -1; i = ne[i]) {
18         int k = e[i];
19         dfs(k);
20         dp[u][0] += max(dp[k][0], dp[k][1]); // 父节点参加时子节点可参加可不参加
21         dp[u][1] += dp[k][0]; // 父节点参加时子节点都不能参加
22     }
23 }
24
25 int main() {
26     memset(h, -1, sizeof h);
27     cin >> n;
28     for (int i = 1; i <= n; i++) {
29         cin >> w[i];
30     }
31     for (int i = 1; i < n; i++) {
32         int a, b; cin >> a >> b;
```

```

33     add(b, a);
34     st[a] = 1; // 标记a有父节点
35 }
36
37 int root = 1;
38 while (st[root]) root++;
39 dfs(root); // 从根节点搜索
40
41 cout << max(dp[root][0], dp[root][1]);
42 }

```

## 皇宫看守

每个节点至少要连接一个被选择的点，并使所有选择的点总花费最小

dp[i, 0] 表示未选择当前点，且选择了其父节点。

dp[i, 1] 表示未选择当前点，且选择了至少一个子节点。

dp[i, 2] 表示选择了当前点。

```

1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  using namespace std;
5  const int N = 1505;
6  int n;
7  int h[N], e[N], ne[N], w[N], idx;
8  int dp[N][N];
9  bool st[N];
10
11 void add(int a, int b){
12     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
13 }
14
15 void dfs(int u){
16     dp[u][2] = w[u];
17     for(int i = h[u]; ~i; i = ne[i]){
18         int k = e[i];
19         dfs(k);
20         dp[u][0] += min(dp[k][1], dp[k][2]);
21         dp[u][2] += min({dp[k][0], dp[k][1], dp[k][2]});
22     }
23     dp[u][1] = 0x3f3f3f3f;
24     for(int i = h[u]; ~i; i = ne[i]){
25         int k = e[i];
26         dp[u][1] = min(dp[u][1], dp[u][0] + dp[k][2] - min(dp[k][1], dp[k][2]));
27     }
28 }
29
30 int main(){
31     memset(h, -1, sizeof h);

```

```

32     cin >> n;
33     for(int i = 1; i <= n; i++){
34         int a, k; cin >> a >> w[a] >> k;
35         while(k--){
36             int b; cin >> b;
37             add(a, b);
38             st[b] = 1;
39         }
40     }
41     int root = 1;
42     while(st[root]) root++;
43     dfs(root);
44     cout << min(dp[root][1], dp[root][2]);
45 }

```

## 树上背包

```

1 //选课 https://www.luogu.com.cn/problem/P2014
2 //O(N*M^2) 有依赖关系的背包问题
3 //对于森林，可以将每一棵树的根节点都接到一个"虚拟根节点0"上，以0作为根节点，dp[0][m+1]即为答案
4 #include <iostream>
5 #include <cstring>
6 using namespace std;
7 const int N = 305;
8 int n, m;
9 int s[N], k[N];
10 int h[N], e[N], ne[N], idx;
11 int dp[N][N]; //选到节点i, 价值不超过j的最大价值
12
13 void add(int a, int b){
14     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
15 }
16
17 void dfs(int u){ //分组背包处理
18     for(int i = h[u]; ~i; i = ne[i]){ //枚举物品组
19         dfs(e[i]);
20         for(int j = m+1; j >= 0; j--){ //枚举体积 j(m+1~0)
21             for(int k = 0; k <= j-1; k++){ //枚举决策 k(0~j-v[u])
22                 dp[u][j] = max(dp[u][j], dp[u][j-k] + dp[e[i]][k]);
23             }
24         }
25     }
26 }

```

```

27
28 int main(){
29     memset(h,-1,sizeof h);
30     cin >> n >> m;
31     for(int i = 1;i <= n;i++){
32         cin >> k[i] >> s[i];
33         for(int j = 1;j <= m;j++) dp[i][j] = s[i]; //初始状态
34         add(k[i],i);
35     }
36     dfs(0);
37     cout << dp[0][m+1];
38 }

```

## 换根DP

树形 DP 中的换根 DP 问题又被称为二次扫描，通常不会指定根结点，并且根结点的变化会对一些值，例如子结点深度和、点权和等产生影响。

通常需要两次 DFS，第一次 DFS 自底向上预处理诸如深度，子树大小，点权和之类的信息，在第二次 DFS 自顶向下开始运行换根动态规划。

```

1 //https://www.luogu.com.cn/problem/P3478
2 //给定一个n个点的树，请求出一个结点，使得以这个结点为根时，所有结点的深度之和最大。
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 2000006;
7 int n;
8 int h[N],e[N],ne[N],idx;
9 long long f[N],siz[N],deep[N];
10
11 void add(int a,int b){
12     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
13 }
14
15 void dfs_d(int u,int fa){
16     siz[u] = 1;
17     deep[u] = deep[fa]+1;
18     for(int i = h[u];~i;i = ne[i]){
19         int k = e[i];
20         if(k == fa) continue;
21         dfs_d(k,u);
22         siz[u] += siz[k];
23     }
24 }
25

```

```

26 void dfs_u(int u,int fa){
27     for(int i = h[u];~i;i = ne[i]){
28         int k = e[i];
29         if(k == fa) continue;
30         f[k] = f[u]-siz[k]+n-siz[k];
31         dfs_u(k,u);
32     }
33 }
34
35 int main(){
36     memset(h,-1,sizeof h);
37     cin >> n;
38     for(int i = 1;i < n;i++){
39         int a,b;cin >> a >> b;
40         add(a,b);add(b,a);
41     }
42
43     dfs_d(1,0);//一般第一遍dfs由底回溯到根统计信息
44     for(int i = 1;i <= n;i++) f[1] += deep[i];//任选一个点(一般为1)作为根
45     dfs_u(1,0);//再dfs由根到底考虑将根不断转移到子节点，更新每个节点作为根时的答案
46
47     long long ans = 0,p = 0;
48     for(int i = 1;i <= n;i++){
49         if(ans < f[i]){ ans = f[i]; p = i; }
50     }
51     cout << p;
52 }

```

```

1 //hdu2196 computer https://acm.hdu.edu.cn/showproblem.php?pid=2196
2 //求每个节点距离其它节点的最远距离
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 20004;
7 int n;
8 int h[N],e[N],ne[N],idx,w[N];
9 int d1[N],d2[N],p1[N],p2[N],up[N];//d1和d2为节点u向下的最大值和次大值,up为向上的最大值
10
11 void add(int a,int b,int c){
12     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
13 }
14
15 void dfs_d(int u,int fa){
16     for(int i = h[u];~i;i = ne[i]){
17         int k = e[i];
18         if(k == fa) continue;
19         dfs_d(k,u);
20         int t = d1[k] + w[i];
21         if(t > d1[u]){

```

```

22         d2[u] = d1[u];d1[u] = t;
23         p2[u] = p1[u];p1[u] = k;
24     }
25     else if(t > d2[u]){
26         d2[u] = t;
27         p2[u] = k;
28     }
29 }
30 }
31
32 void dfs_u(int u,int fa){
33     for(int i = h[u];~i;i = ne[i]){
34         int k = e[i];
35         if(k == fa) continue;
36         if(p1[u] == k){ up[k] = max(up[u],d2[u]) + w[i]; }
37         else { up[k] = max(up[u],d1[u]) + w[i];}
38         dfs_u(k,u);
39     }
40 }
41
42 int main(){
43     while(cin >> n){
44         for(int i = 1;i <= n;i++){
45             h[i] = -1; d1[i] = d2[i] = up[i] = 0;
46         }
47         idx = 0;
48
49         for(int i = 2;i <= n;i++){
50             int b,c;cin >> b >> c;
51             add(i,b,c);add(b,i,c);
52         }
53         dfs_d(1,0);
54         dfs_u(1,0);
55
56         for(int i = 1;i <= n;i++){
57             int now = max(d1[i],up[i]);
58             cout << now << '\n';
59         }
60     }
61 }

```

## 概率DP

一般情况下，解决概率问题顺序循环，解决期望问题逆序循环。



# 概率DP

[Problem - 148D - Codeforces](#)

袋子里有  $i$  只白鼠和  $j$  只黑鼠，公主和龙轮流从袋子里抓老鼠。谁先抓到白色老鼠谁就赢，如果袋子里没有老鼠了并且没有谁抓到白色老鼠，那么算龙赢。公主每次抓一只老鼠，龙每次抓完一只老鼠之后会有一只老鼠跑出来。每次抓的老鼠和跑出来的老鼠都是随机的。公主先抓。问公主赢的概率。

设  $f_{i,j}$  为轮到公主时袋子里有  $i$  只白鼠， $j$  只黑鼠，公主赢的概率。初始化边界， $f_{0,j} = 0$  因为没有白鼠了算龙赢， $f_{i,0} = 1$  因为抓一只就是白鼠，公主赢。

考虑  $f_{i,j}$  的转移：

1. 公主抓到一只白鼠，公主赢了。概率为  $\frac{i}{i+j}$ ；
2. 公主抓到一只黑鼠，龙抓到一只白鼠，龙赢了。概率为  $\frac{j}{i+j} \cdot \frac{i}{i+j-1}$ ；
3. 公主抓到一只黑鼠，龙抓到一只黑鼠，跑出来一只黑鼠，转移到  $f_{i,j-3}$ 。概率为  $\frac{j}{i+j} \cdot \frac{j-1}{i+j-1} \cdot \frac{j-2}{i+j-2}$ ；
4. 公主抓到一只黑鼠，龙抓到一只黑鼠，跑出来一只白鼠，转移到  $f_{i-1,j-2}$ 。概率为  $\frac{j}{i+j} \cdot \frac{j-1}{i+j-1} \cdot \frac{i}{i+j-2}$ ；

考虑公主赢的概率，第二种情况不参与计算。并且要保证后两种情况合法，所以还要判断  $i, j$  的大小，满足第三种情况至少要有 3 只黑鼠，满足第四种情况要有 1 只白鼠和 2 只黑鼠。

```
1  #include <bits/stdc++.h>
2
3  const int N = 1003;
4  double dp[N][N];
5
6  int main(){
7      int w,b; std::cin >> w >> b;
8      for(int i = 1; i <= w; i++) dp[i][0] = 1;
9      for(int j = 1; j <= b; j++) dp[0][j] = 0;
10
11     for(int i = 1; i <= w; i++){
12         for(int j = 1; j <= b; j++){
13             dp[i][j] += (double)i/(i+j); //情况1
14             if(j >= 3) { //情况3
15                 dp[i][j] += (double)j/(i+j) * (j-1)/(i+j-1) * (j-2)/(i+j-2) *
dp[i][j-3];
16             }
17             if(i >= 1 && j >= 2) { //情况4
18                 dp[i][j] += (double)j/(i+j) * (j-1)/(i+j-1) * i/(i+j-2) * dp[i-1][j-2];
19             }
20         }
21     }
22
23     printf("%.91f", dp[w][b]);
24 }
```

## 期望DP

### [UVA12369 Cards - 洛谷](#)

给定一副扑克牌，求翻出A张黑桃，B张红桃，C张梅花，D张方块需要的期望次数。其中大小王可以当作任意花色使用。

$dp[a][b][c][d][x][y]$  表示该状态走到终点状态的翻牌数的期望数

a张黑桃，b张红桃，c张梅花，d张方块，小v/大王状态为x/y(1/2/3/4表示放到a/b/c/d，0表示没翻出来)

```
1  #include <iostream>
2  #include <cstring>
3
4  const int N = 14;
5  const double INF = 1e18;
6  int A,B,C,D;
7  double dp[N][N][N][N][5][5];
8
9  double dfs(int a,int b,int c,int d,int x,int y){
10     auto &v = dp[a][b][c][d][x][y];
11     if(v >= 0) return v;
12     // 当前已有的牌堆
13     int sa = a + (x == 1) + (y == 1);
14     int sb = b + (x == 2) + (y == 2);
15     int sc = c + (x == 3) + (y == 3);
16     int sd = d + (x == 4) + (y == 4);
17
18     if(sa >= A && sb >= B && sc >= C && sd >= D) return v = 0; // 终点状态
19
20     int rest = 54 - (sa + sb + sc + sd); // 剩余卡牌总数量
21     if(rest <= 0) return v = INF; // 卡池已经抽空,但仍然未达目标状态
22
23     v = 1; // 初始化为1
24     // 四种花色
25     if(a < 13) v += (13.0-a)/rest * dfs(a+1,b,c,d,x,y); // 概率*期望
26     if(b < 13) v += (13.0-b)/rest * dfs(a,b+1,c,d,x,y);
27     if(c < 13) v += (13.0-c)/rest * dfs(a,b,c+1,d,x,y);
28     if(d < 13) v += (13.0-d)/rest * dfs(a,b,c,d+1,x,y);
29
30     if(x == 0) { // 小王,仅未抽中才转移
31         double t = INF;
32         for(int i = 1; i <= 4; i++) {
33             t = std::min(t, 1.0/rest * dfs(a,b,c,d,i,y));
34         }
35         v += t;
36     }
37     if(y == 0) { // 大王
38         double t = INF;
39         for(int i = 1; i <= 4; i++){
40             t = std::min(t, 1.0/rest * dfs(a,b,c,d,x,i));
```

```

41     }
42     v += t;
43 }
44 return v;
45 }
46
47 void sol(){
48     std::cin >> A >> B >> C >> D;
49     std::memset(dp,-1,sizeof dp);
50
51     double ans = dfs(0,0,0,0,0,0);
52     if(ans > INF/2) ans = -1;
53     printf("%.31f\n",ans);
54 }
55
56 int main(){
57     int t; std::cin >> t;
58     for(int i = 1;i <= t;i++){
59         printf("Case %d: ",i);
60         sol();
61     }
62 }

```

## 记忆化搜索

记忆化搜索是一种通过记录已经遍历过的状态的信息，从而避免对同一状态重复遍历的搜索实现方式。

因为记忆化搜索确保了每个状态只访问一次，它也是一种常见的动态规划实现方式。

### 滑雪

```

1 //https://ac.nowcoder.com/acm/problem/235954
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 305;
6 int n, m;
7 int a[N][N];
8 int dp[N][N]; //dp[i][j]表示从i,j开始滑的最长路径
9 int px[] = { 0,0,-1,1 }, py[] = { 1,-1,0,0 };
10
11 int dfs(int x, int y) {
12     if (dp[x][y]) return dp[x][y]; //如果已经计算过了，就可以直接返回答案，避免重复计算
13     dp[x][y] = 1; //注意dp[x][y]至少为1(四个方向都不能滑)
14     for (int i = 0; i < 4; i++) { //枚举上下左右四个方向
15         int dx = x + px[i], dy = y + py[i];

```

```

16         if (dx >= 1 && dx <= n && dy >= 1 && dy <= m && a[x][y]>a[dx][dy]) { //判
断能否从x,y滑向滑dx,dy
17             dp[x][y] = max(dp[x][y], dfs(dx, dy) + 1); //更新dp[x][y]
18         }
19     }
20     return dp[x][y];
21 }
22
23 int main() {
24     cin >> n >> m;
25     for (int i = 1; i <= n; i++) {
26         for (int j = 1; j <= m; j++) {
27             cin >> a[i][j];
28         }
29     }
30
31     int ans = 0; //因为可以在任意一点开始滑，所以要遍历一遍滑雪场，取最大值
32     for (int i = 1; i <= n; i++) {
33         for (int j = 1; j <= m; j++) {
34             ans = max(ans, dfs(i, j));
35         }
36     }
37     cout << ans;
38 }

```

## DP优化

### 单调队列优化

- 加入所需元素：向单调队列重复加入元素直到当前元素达到所求区间的右边界，这样就能保证所需元素都在单调队列中。
- 弹出越界队首：单调队列本质上是维护的是所有已插入元素的最值，但我们想要的往往是一个区间最值。于是我们弹出在左边界外的元素，以保证单调队列中的元素都在所求区间中。
- 获取最值：直接取队首作为答案即可。

```

1 //烽火传递 https://loj.ac/p/10180
2 //给定数组a[N],每连续m个元素至少有一个被选中,最小选中代价
3 #include <iostream>
4 using namespace std;
5 const int N = 200005;
6 int n,m,a[N];
7 int dp[N]; //dp[i]表示选择第i个时的最小代价
8 int q[N],hh,tt;
9
10 int main(){
11     cin >> n >> m;

```

```

12     for(int i = 1;i <= n;i++) cin >> a[i];
13
14     for(int i = 1;i <= n;i++){
15         dp[i] = dp[q[hh]] + a[i];//队首即为最值 dp[i] = min(dp[i-m]~dp[i-1]) +
a[i]
16         if(hh <= tt && i - q[hh] >= m) hh++;//弹出越界队首
17         while(hh <= tt && dp[i] <= dp[q[tt]]) tt--;//加入所需元素,并保持队列单调递增
18         q[++tt] = i;
19     }
20
21     int ans = 0x3f3f3f3f;
22     for(int i = n-m+1;i <= n;i++) ans = min(ans,dp[i]);
23     cout << ans;
24 }

```

```

1 //修剪草坪 https://loj.ac/p/10177
2 //给定a[N],不能选择连续m个元素,求最大代价
3 //dp[i] = max(dp[i-1],max(dp[j-1]+sum(j+1,i))) //选/不选第i个
4 //其中dp[j-1]+sum(j+1,i) = dp[j-1] + s[i] - s[j] 只需要单调队列求出最大的dp[j-1]-
s[j]
5 #include <iostream>
6 using namespace std;
7 const int N = 100005;
8 int n,m;
9 long long a[N],dp[N];
10 int q[N],hh,tt;
11
12 long long g(int i){
13     return dp[i-1] - a[i];
14 }
15
16 int main(){
17     cin >> n >> m;
18     for(int i = 1;i <= n;i++){
19         cin >> a[i]; a[i] += a[i-1];
20     }
21
22     for(int i = 1;i <= n;i++){
23         dp[i] = max(dp[i-1],g(q[hh]) + a[i]);
24         //区间[i-m,i-1],窗口大小为m (不包括i)
25         if(hh <= tt && i - q[hh] >= m) hh++;
26         //区间[i-m+1,i-1],窗口大小为m-1 (不包括i)
27         while(hh <= tt && g(i) >= g(q[tt])) tt--;
28         q[++tt] = i;
29         //区间[i-m+1,i],窗口大小为m (包括i)
30     }
31     cout << dp[n];
32 }

```

## 数据结构优化

[线段树优化 P4644 \(luogu.com.cn\)](#)

给定 $n$ 个区间以及其花费 $\{[l, r], c\}$ , 选择合理的区间使其能够完全覆盖区间 $[st, ed]$ , 并使得总花费最小

将所有区间按右端点排序后, 遍历 $n$ 个区间, 对于当前区间 $l, r, c$

$dp[r] = \min(dp[r], dp[l-1 \sim r-1] + c)$

考虑线段树优化, 单点修改, 求区间最小值

```
1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  using namespace std;
5  const int N = 100005;
6  const long long INF = 0x3f3f3f3f3f3f3f3f;
7  int n, st, ed;
8  long long dp[N];
9
10 struct node{
11     int l, r;
12     long long c;
13     bool operator < (const auto &e2){
14         return r < e2.r;
15     }
16 }a[N];
17
18 struct ST{
19     int l, r;
20     long long dat;
21 }t[N<<2];
22
23 void pushup(ST &p, ST &p1, ST &pr){
24     p.dat = min(p1.dat, pr.dat);
25 }
26
27 void pushup(int p){
28     pushup(t[p], t[p<<1], t[p<<1|1]);
29 }
30
31 void build(int p, int l, int r){
32     t[p] = {l, r};
33     if(l == r){
34         t[p].dat = INF;
35         return;
36     }
37     int mid = l + r >> 1;
```

```

38     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
39     pushup(p);
40 }
41
42 void modify(int p,int l,int r,long long x){
43     if(l <= t[p].l && r >= t[p].r){
44         t[p].dat = x;
45         return;
46     }
47     int mid = t[p].l + t[p].r >> 1;
48     if(l <= mid) modify(p<<1,l,r,x);
49     if(r > mid) modify(p<<1|1,l,r,x);
50     pushup(p);
51 }
52
53 ST query(int p,int l,int r){
54     if(l <= t[p].l && r >= t[p].r){
55         return t[p];
56     }
57     int mid = t[p].l + t[p].r >> 1;
58     if(r <= mid) return query(p<<1,l,r);
59     if(l > mid) return query(p<<1|1,l,r);
60     ST pl = query(p<<1,l,r),pr = query(p<<1|1,l,r),ans;
61     pushup(ans,pl,pr);
62     return ans;
63 }
64
65 int main(){
66     cin >> n >> st >> ed;
67     build(1,0,ed);
68     for(int i = 1;i <= n;i++){
69         cin >> a[i].l >> a[i].r >> a[i].c;
70     }
71     sort(a+1,a+n+1);
72     memset(dp,0x3f,sizeof dp);
73
74     for(int i = 1;i <= n;i++){
75         auto [l,r,c] = a[i];
76         if(l - 1 <= st - 1) dp[r] = min(dp[r],c);
77         else dp[r] = min(dp[r],query(1,l-1,r-1).dat + c);
78         modify(1,r,r,dp[r]);
79     }
80
81     if(dp[ed] == INF) cout << -1;
82     else cout << dp[ed];
83 }

```

在长度为n的数列a[]中，求长度为m的严格上升子序列的个数

dp[i,j]表示以a[i]结尾，且长度为j的上升子序列个数

$dp[i,j] = \sum dp[k,j-1]$  其中  $1 \leq k < i$  &&  $a[k] < a[i]$

考虑将a[]离散化后建立树状数组，query(id(a[i])-1)为1~id(a[i])中的 $\sum dp[k,j-1]$ ，插入的先后顺序保证了不会重复计算到后面的值

```
1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  using namespace std;
5  const int N = 1005, mod = 1e9+7;
6  int n, m;
7  int a[N], hs[N];
8  long long t[N], dp[N][N];
9
10 void add(int p, long long x){
11     while(p <= n){
12         t[p] = (t[p] + x)%mod;
13         p += p&-p;
14     }
15 }
16
17 long long query(int p){
18     long long ans = 0;
19     while(p > 0){
20         ans = (ans + t[p]) % mod;
21         p -= p&-p;
22     }
23     return ans;
24 }
25
26 void sol(){
27     memset(dp, 0, sizeof dp);
28     cin >> n >> m;
29
30     for(int i = 1; i <= n; i++){
31         scanf("%d", &a[i]); hs[i] = a[i];
32     }
33     sort(hs+1, hs+n+1);
34     for(int i = 1; i <= n; i++){ a[i] = lower_bound(hs+1, hs+n+1, a[i]) - hs; }
35
36     for(int i = 1; i <= n; i++){ dp[i][1] = 1; }
37
38     for(int j = 2; j <= m; j++){
39         memset(t, 0, sizeof t);
40         for(int i = 1; i <= n; i++){
41             dp[i][j] = query(a[i]-1);
42             add(a[i], dp[i][j-1]);
43         }
44     }
45 }
```



```

44     }
45     long long ans = 0;
46     for(int i = 1; i <= n; i++){
47         ans = (ans + dp[i][m]) % mod;
48     }
49     cout << ans << '\n';
50 }
51
52 int main(){
53     int T; cin >> T;
54     for(int o = 1; o <= T; o++){
55         cout << "Case #" << o << ": "; sol();
56     }
57 }

```

## 斜率优化

[【学习笔记】动态规划—斜率优化DP（超详细）——辰星凌的博客QAQ](#)

先写出DP方程，例如： $dp[i] = \min\{dp[j] + (s[i] - s[j])^2 + m\}$

去掉 $\min$ ，对齐进行移项变换为 $y = kx + b$ 的点斜式。

把含有 $function(i) * function(j)$ 的表达式看做斜率 $k$ 乘以未知数 $x$ ，含有 $dp[i]$ 或常数项在 $b$ 的表达式中，含有 $function(j)$ 的项在 $y$ 的表达式中。

$$dp[j] + s[j]^2 = 2 * s[i] * s[j] + dp[i] - s[i]^2 - m。$$

此处 $y = dp[j] + s[j]^2$ ， $k = 2 * s[i]$ ， $x = s[j]$ ， $b = dp[i] - s[i]^2 - m。$

即找出一个点 $(s[j], dp[j] + s[j]^2)$ ，使得 $b$ 也即 $dp[i]$ 最大

写出DP方程后，判断能否使用斜率优化，即是否存在 $function(i) * function(j)$ 的项，或者

$$\frac{Y(j)-Y(j')}{X(j)-X(j')}$$

通过大小于符号或者换 $b$ 中 $dp[i]$ 的符号结合题目要求( $\min/\max$ )判断是上凸包还是下凸包。

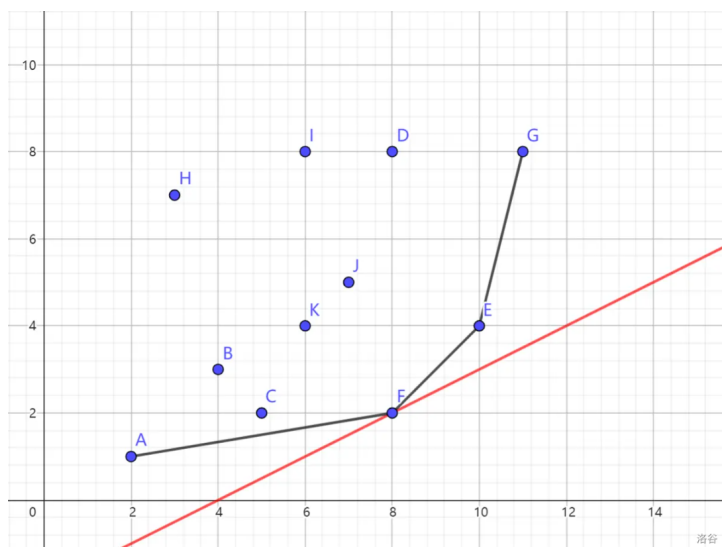
[P10979 任务安排 2 - 洛谷\(luogu.com.cn\)](#)

$$dp[i] = \min_{0 \leq j < i} \{dp[j] - (s + st[i]) * sc[j]\} + st[i] * sc[i] + s * sc[n]$$

把 $\min$ 函数去掉，把关于 $j$ 的值 $dp[j]$ 和 $sc[j]$ 看作变量，其余部分看作常数得到：

$$dp[j] = (s + st[i]) * sc[j] + dp[i] - st[i] * sc[i] - s * sc[n]$$

以 $sc[j]$ 为横坐标， $dp[j]$ 为纵坐标建立平面直角坐标系，当前点 $i$ 斜率 $k$ 为固定值 $s + st[i]$ ，决策候选集合是坐标系中的一个点集，每个决策 $j$ 都对应着坐标系中的一个点 $(sc[j], dp[j])$ ，要使 $dp[i]$ 最小，也就是在前 $i - 1$ 个点中选择一个点，使得在 $k$ 一定的情况下，截距最小，不难发现，答案必然在点集的一个右下凸包上：



由于本题中 $t$ 为正整数， $st$ 递增也即斜率 $k$ 递增，每次决策时将队首斜率小于 $k$ 的弹出，队头即为目标 $j$ 。将点 $i$ 插入前，若队尾两点和点 $i$ 形成凹包，则不断将队尾弹出，直到满足凸包。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int N = 300005;
5  long long n,s;
6  long long dp[N];
7  long long c[N],t[N],sc[N],st[N];
8  int q[N],hh,tt;
9
10 long long X(int i){return sc[i];}
11 long long Y(int i){return dp[i];}
12 long long up(int a,int b) {return Y(a) - Y(b);} //分子
13 long long down(int a,int b) {return X(a) - X(b);} //分母
14
15
16 int main(){
17     cin >> n >> s;
18     for(int i = 1;i <= n;i++){
19         cin >> t[i] >> c[i];
20         st[i] = st[i-1] + t[i];
21         sc[i] = sc[i-1] + c[i];
22     }
23
24     for(int i = 1;i <= n;i++){
25         int k = st[i] + s;
26         while(hh <= tt-1 && up(q[hh+1],q[hh]) <= k * down(q[hh+1],q[hh])) hh++;
27         int j = q[hh];
28         dp[i] = dp[j] - (s+st[i])*sc[j] + st[i]*sc[i] + s*sc[n];
29         while(hh <= tt-1 && up(i,q[tt])*down(q[tt],q[tt-1]) <= up(q[tt],q[tt-1])*down(i,q[tt]))tt--;
30         q[++tt] = i;
31     }
32     cout << dp[n];

```

[P5785 [SDOI2012\] 任务安排 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P5785)

本题，若 $k$ 不满足单调性，则需要维护整个凸包，每次决策时只需要在队列中用 二分/CDQ/平衡树 找到点 $j$ 满足：点 $j$ 左边的斜率都小于 $k$ ，右边的斜率都大于 $k$ 。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int N = 300005;
5  long long n,s;
6  long long dp[N];
7  long long c[N],t[N],sc[N],st[N];
8  int q[N],hh,tt;
9
10 struct node{
11     long long x,y;
12 };
13
14 int lb(long long k,int bl,int br){
15     auto check = [&](int x)->bool{
16         node p1 = {sc[q[x]],dp[q[x]]},p2 = {sc[q[x+1]],dp[q[x+1]]};
17         return (p2.y - p1.y) >= k*(p2.x - p1.x);
18     };
19     int l = bl,r = br;
20     while(l < r){
21         int mid = l + r >> 1;
22         if(check(mid)) r = mid;
23         else l = mid + 1;
24     }
25     return q[r];
26 }
27
28 int main(){
29     cin >> n >> s;
30     for(int i = 1;i <= n;i++){
31         cin >> t[i] >> c[i];
32         st[i] = st[i-1] + t[i];
33         sc[i] = sc[i-1] + c[i];
34     }
35
36     for(int i = 1;i <= n;i++){
37         int j = lb(st[i]+s,hh,tt);
38         dp[i] = dp[j] - (s+st[i])*sc[j] + st[i]*sc[i] + s*sc[n];
39         while(hh <= tt - 1){
40             node p1 = {sc[q[tt-1]],dp[q[tt-1]]},p2 = {sc[q[tt]],dp[q[tt]]},p3 =
{sc[i],dp[i]};
41             if(__int128(p3.y-p2.y)*(p2.x-p1.x) <= __int128(p2.y-p1.y)*(p3.x-
p2.x)) tt--;

```

```

42         else break;
43     }
44     q[++tt] = i;
45 }
46 cout << dp[n];
47 }

```

### 303. 运输小猫 - AcWing题库

对于每只小猫所需要的早出发时间为  $a[i] = t[i] - \sum d[i]$ , 排序后对  $a[]$  建立前缀和数组  $s[]$

令  $dp[i][j]$  表示前  $i$  个人运输前  $j$  只小猫所需要的最小等待次数。第  $i$  个人运输第  $k+1 \sim j$  只猫。则有：

$$dp[i][j] = \min\{dp[i-1][k] + a[j] * (j - k) - (s[j] - s[k])\}$$

去掉min, 移项得：

$$dp[i-1][k] + s[k] = a[j] * k + dp[i][j] - a[j] * j + s[j]$$

斜率优化  $O(1)$  求出  $k$ , 以  $k$  为横轴,  $dp[i-1][k] + s[k]$  为纵轴建立坐标系, 当前斜率为  $a[j]$ , 坐标轴上每个点为  $(k, dp[i-1][k] + s[k])$

```

1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  using namespace std;
5  const int N = 100005;
6  int n,m,p;
7  long long d[N];
8  long long h[N],t[N],a[N],s[N];
9  long long dp[105][N];
10 int q[N],hh,tt;
11
12 struct node{
13     long long x,y;
14 };
15
16 int main(){
17     cin >> n >> m >> p;
18     for(int i = 2;i <= n;i++){
19         cin >> d[i];
20         d[i] += d[i-1];
21     }
22     for(int i = 1;i <= m;i++){
23         cin >> h[i] >> t[i];
24         a[i] = t[i] - d[h[i]];
25     }
26     sort(a+1,a+m+1);
27

```

```

28     for(int i = 1; i <= m; i++){
29         s[i] = s[i-1] + a[i];
30     }
31
32     memset(dp, 0x3f, sizeof dp);
33     dp[0][0] = 0;
34     for(int i = 1; i <= p; i++){
35         hh = 0, tt = 0;
36         for(int j = 1; j <= m; j++){
37             while(hh <= tt - 1){//由于斜率a[j]递增,每次维护队首即可
38                 node p1 = {q[hh], dp[i-1][q[hh]]+s[q[hh]]}, p2 = {q[hh+1], dp[i-1]
[q[hh+1]]+s[q[hh+1]]};
39                 if((p2.y-p1.y) <= a[j]*(p2.x-p1.x)) hh++;
40                 else break;
41             }
42             dp[i][j] = dp[i-1][q[hh]] + a[j]*(j-q[hh]) - s[j] + s[q[hh]];
43             while(hh <= tt-1){
44                 node p1 = {q[tt-1], dp[i-1][q[tt-1]]+s[q[tt-1]]}, p2 =
{q[tt], dp[i-1][q[tt]]+s[q[tt]]};
45                 node p3 = {j, dp[i-1][j]+s[j]}; //插入的点p3,取值应为dp[i-1]
46                 if((p3.y-p2.y)*(p2.x-p1.x) <= (p2.y-p1.y)*(p3.x-p2.x)) tt--;
47                 else break;
48             }
49             q[++tt] = j;
50         }
51     }
52     cout << dp[p][m];
53 }

```

## 滚动数组

简要说就是通过观察dp方程来判断需要使用哪些数据,可以抛弃哪些数据,一旦找到关系,就可以用新的数据不断覆盖旧的没用的数据来**优化空间**。还要注意**根据数据是否可以直接继承考虑是否需要清空数组**

### 利用对自然数取模的周期性

```

1 //例: 递推求斐波那契数列
2 for (int i=3; i<=n; i++) f[i%3]=f[(i-1)%3]+f[(i-2)%3];
3 cout << f[n%3];

```

```

1 dp[i][s1][s2] = max(dp[i][s1][s2], dp[i-1&1][s2][s3]+1);
2 //只用到了i-1项,可以优化为
3 dp[i&1][s1][s2] = max(dp[i&1][s1][s2], dp[i-1&1][s2][s3]+1);

```

```

1  for(int i = 1;i <= n+1;i++){//dp[N][M]
2      for(auto s1:st[i-1]){
3          for(auto s2:st[i]){
4              if(s1&s2) continue;
5              dp[i][s2] = (dp[i][s2]+dp[i-1][s1])%mod;
6          }
7      }
8  }
9  //可以优化为
10 for(int i = 1;i <= n+1;i++){//dp[2][M]
11     for(auto s1:st[i-1]){
12         for(auto s2:st[i]){
13             if(s1&s2) continue;
14             dp[i&1][s2] = 0;
15         }
16     }
17     for(auto s1:st[i-1]){
18         for(auto s2:st[i]){
19             if(s1&s2) continue;
20             dp[i&1][s2] = (dp[i&1][s2]+dp[i-1&1][s1])%mod;
21         }
22     }
23 }

```

## 其他

### 环形处理

断环成链，复制拼接

[P10957 环路运输 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10957)

```

1  #include <iostream>
2  using namespace std;
3  const int N = 2000006;
4  int n;
5  long long a[N];
6  int q[N],hh,tt;
7
8  long long g(int i){
9      return a[i] - i;
10 }
11
12 int main(){
13     cin >> n;

```

```

14     for(int i = 1;i <= n;i++){
15         cin >> a[i];
16         a[i+n] = a[i];
17     }
18
19     long long ans = 0;
20     for(int i = 1;i <= n << 1;i++){//单调队列优化
21         ans = max(ans,a[i] + i + g(q[hh]));
22         if(hh <= tt && i - q[hh] >= n/2) hh++;
23         while(hh <= tt && g(i) >= g(q[tt])) tt--;
24         q[++tt] = i;
25     }
26     cout << ans;
27 }

```

## 二次DP

一次选择N和1断开，另一次选择N和1连接。答案取两次DP的最值

[P6064 [USACO05JAN\] Naptime G - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P6064)

```

1  #include <iostream>
2  #include <algorithm>
3  #include <cstring>
4  using namespace std;
5  const int N = 3900;
6  int n,a[N],m;
7  int dp[2][N][2];
8
9  int main(){
10     cin >> n >> m;
11     for(int i = 1;i <= n;i++){
12         cin >> a[i];
13     }
14
15     memset(dp,-0x3f,sizeof dp);
16     for(int i = 1;i <= n;i++) dp[i&1][0][0] = 0;
17     dp[1&1][1][1] = 0;
18     for(int i = 2;i <= n;i++){
19         dp[i&1][0][0] = dp[(i-1)&1][0][0];
20         for(int j = 1;j <= m;j++){
21             dp[i&1][j][0] = max(dp[(i-1)&1][j][0],dp[(i-1)&1][j][1]);
22             dp[i&1][j][1] = max(dp[(i-1)&1][j-1][0],dp[(i-1)&1][j-1][1]+a[i]);
23         }
24     }
25     int ans = max(dp[n&1][m][0],dp[n&1][m][1]);
26
27     memset(dp,-0x3f,sizeof dp);
28     for(int i = 1;i <= n;i++) dp[i&1][0][0] = 0;
29     dp[1][1][1] = a[1];

```

```

30     for(int i = 2;i <= n;i++){
31         for(int j = 1;j <= m;j++){
32             dp[i&1][j][0] = max(dp[(i-1)&1][j][0],dp[(i-1)&1][j][1]);
33             dp[i&1][j][1] = max(dp[(i-1)&1][j-1][0],dp[(i-1)&1][j-1][1]+a[i]);
34         }
35     }
36     ans = max(ans,dp[n&1][m][1]);
37     cout << ans;
38 }

```

## 状态机DP

### [4747. DNA序列 - AcWing题库](#)

给定m个DNA序列片段，计算有多少长度为n的DNA序列不包含上述m个片段

$0 \leq m \leq 10, 1 \leq n \leq 2 * 10^9$

```

1  #include <iostream>
2  #include <queue>
3  #include <cstring>
4  #include <vector>
5
6  namespace ACAM{
7      const int N = 105,M = 4;
8      int son[N][M],cnt[N],fail[N],idx;
9
10     struct Trie{
11
12         Trie(){idx = 0;init(idx);};
13
14         void init(int p){
15             fail[p] = cnt[p] = 0;
16             std::memset(son[p],0,sizeof son[p]);
17         }
18
19         int get(char x){
20             if(x == 'A') return 0;
21             if(x == 'C') return 1;
22             if(x == 'T') return 2;
23             if(x == 'G') return 3;
24             return 0;
25         }
26
27         void insert(const std::string &s){
28             int p = 0;
29             for(int i = 0;i < s.size();i++){
30                 int u = get(s[i]);
31                 if(!son[p][u]){

```



```

32         son[p][u] = ++idx;
33         init(idx);
34     }
35     p = son[p][u];
36 }
37 cnt[p]++;
38 }
39
40 void get_fail(){
41     std::queue<int>q;
42     for(int u = 0;u < M;u++){
43         if(son[0][u]) {
44             fail[son[0][u]] = 0;
45             q.push(son[0][u]);
46         }
47     }
48     while(q.size()){
49         int p = q.front();
50         q.pop();
51         cnt[p] += cnt[fail[p]]; //将当前节点的禁止状态与fail指针的禁止状态合并
52         for(int u = 0;u < M;u++){
53             if(son[p][u]) {
54                 fail[son[p][u]] = son[fail[p]][u];
55                 q.push(son[p][u]);
56             }
57             else{
58                 son[p][u] = son[fail[p]][u];
59             }
60         }
61     }
62 }
63 };
64 }
65 using ACAM::Trie;
66 using ACAM::fail,ACAM::son,ACAM::cnt,ACAM::idx;
67
68 const int mod = 100000;
69 template<typename T>
70 struct Mat{
71     int n;
72     std::vector<std::vector<T>>>a;
73
74     Mat(int _n,T val){
75         n = _n;
76         a = std::vector<std::vector<T>>>(n,std::vector<T>(n,val));
77     }
78
79     Mat<T> operator * (const Mat<T> &m2){
80         Mat<T> ans(n,0);
81         for(int i = 0;i < n;i++){
82             for(int j = 0;j < n;j++){
83                 for(int k = 0;k < n;k++){

```

```

84         ans.a[i][j] = (ans.a[i][j] + a[i][k] * m2.a[k][j]) % mod;
85     }
86 }
87 }
88 return ans;
89 }
90
91 void norm(){
92     for(int i = 0; i < n; i++) a[i][i] = 1;
93 }
94
95 Mat<T> qmi(long long b){
96     auto base = *this;
97     Mat<T> ans(n,0);
98     ans.norm();
99     while(b){
100         if(b&1) ans = ans * base;
101         b >>= 1;
102         base = base * base;
103     }
104     return ans;
105 }
106 };
107
108 const int N = 12;
109 std::string s[N];
110
111 int main(){
112     int m,n; std::cin >> m >> n;
113     Trie t;
114     for(int i = 1; i <= m; i++){
115         std::cin >> s[i];
116         t.insert(s[i]);
117     }
118     t.get_fail();
119
120     Mat<long long> base(idx+1,0);
121
122     for(int i = 0; i <= idx; i++){
123         for(int u = 0; u < 4; u++){
124             int p = son[i][u];
125             if(cnt[p]) continue;
126             base.a[p][i]++; //状态i通过字符c转移至状态p //dp[k][p] += dp[k-1][i]
127         }
128     }
129
130     base = base.qmi(n); //矩阵快速幂加速递推
131
132     long long res = 0;
133     for(int i = 0; i <= idx; i++){
134         res = (res + base.a[i][0]) % mod;
135     }

```

```
136 |  
137 |     std::cout << res;  
138 | }
```

---